TECNOLOGÍAS PARA LA AUTOMATIZACIÓN · 2º PARCIAL ROBÓTICA

Guías del profe

Las 7 guías de clase del robot 3R en un solo archivo, en el orden en que se dieron

Qué es esto

Son las guías que el profe subió en las clases 5 a 9, convertidas a PDF y unidas en orden. Explican cada rutina de Octave/Matlab que se usó en clase. Para estudiar rápido, empezá por el **resumen con ejercicios modelo**; usá estas guías para profundizar un tema puntual.

#	Guía	Tema	Clase
1	**Robot 3R: cinemática y generación de Q**	CD y CI geométrica del 3R, matriz Q de poses, dibujar el robot	5
2	**De la recta 2D a una trayectoria cartesiana**	Recta paramétrica P1→P2 + CI punto a punto	6
3	**De la ecuación del círculo a la trayectoria**	Círculo paramétrico en 3D + CI	7
4	**Cuadrado y animación del robot 3R**	Trayectoria cuadrada, animación	9
5	**Jacobiano simbólico del robot 3R**	Derivadas, forma abreviada, factor π/180	9
6	**Programa de prueba de CI numérica 3R**	El programa línea por línea	9
7	**Relación entre CI y DLS**	$\Delta Q = J^T(JJ^T+\lambda^2 I)^{-1}e$, λ, α, tolerancia, ciclo completo	9

Ojo: en estas guías el robot 3R usa metros, λ = 0,01 y tolerancia 1e-5 m. Las funciones generales del 2º parcial (`cinematica_inversa`, etc.) usan mm, λ = 0,05 y tolerancia 1e-3 mm.

Robot 3R: Cinemática y generación de configuraciones articulares

Guía didáctica para el uso de las rutinas MATLAB

Esta guía presenta, de manera progresiva, las rutinas desarrolladas para un robot espacial de tres grados de libertad (3R). El objetivo es distinguir con claridad qué problema resuelve cada programa, qué datos se introducen, qué resultados entrega y cómo se relacionan las distintas rutinas.

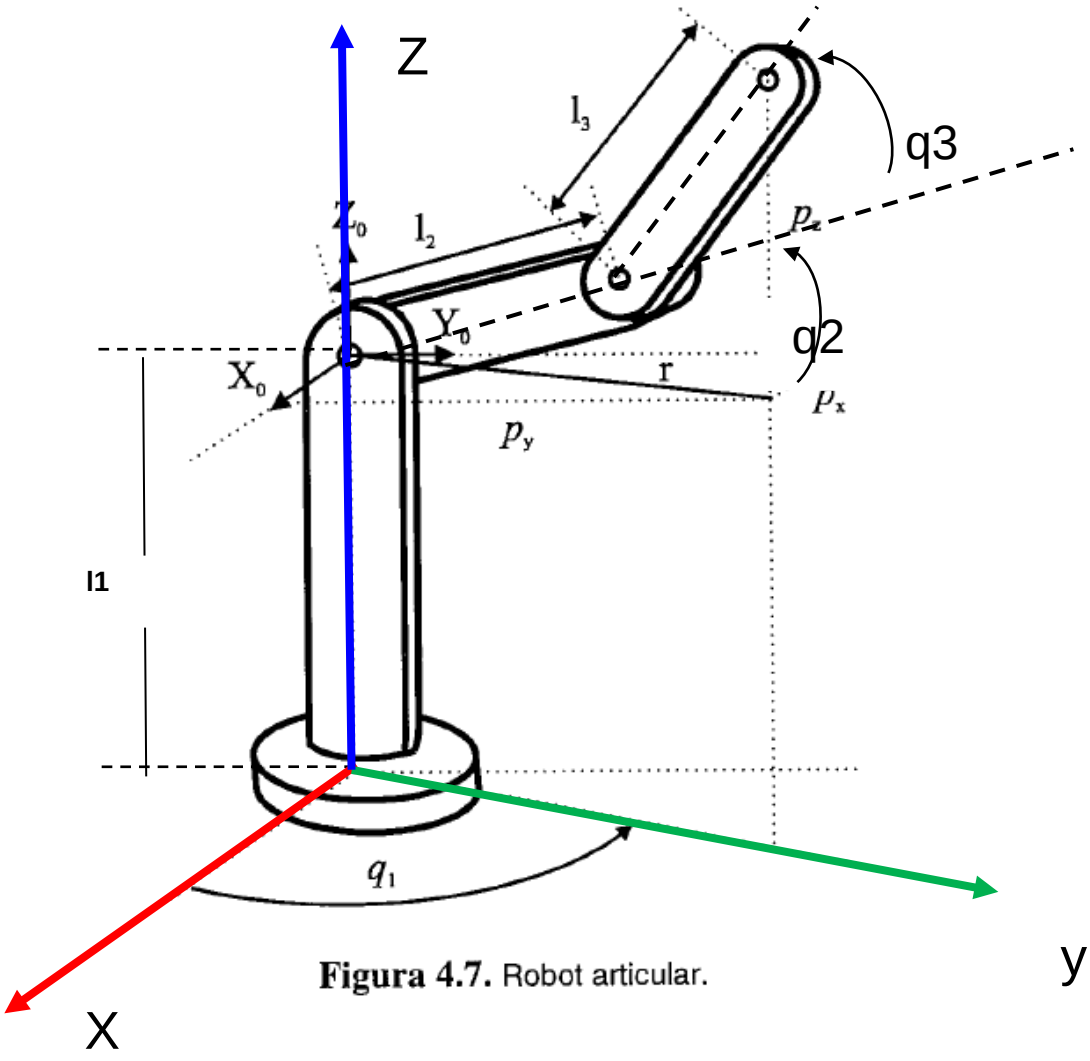


Figura 4.7. Robot articular.

Idea general: q1,q2,q3 ↔ posición cartesiana x,y,z

1. Cinemática de un robot: idea general

En robótica, la cinemática estudia la relación geométrica entre las variables articulares del robot y la posición de su efector final, sin analizar todavía fuerzas ni torques. En nuestro robot, las variables articulares son los tres ángulos q1, q2 y q3.

1.1 Cinemática directa

La cinemática directa responde a la pregunta: si conozco los ángulos de las articulaciones, ¿dónde queda el efector final?

ENTRADA: q1, q2, q3 → CINEMÁTICA DIRECTA → SALIDA: x, y, z

Es decir, el espacio articular se transforma en espacio cartesiano. En este material todos los ángulos se expresan en grados y las posiciones en metros.

1.2 Cinemática inversa

La cinemática inversa resuelve el problema opuesto: si se desea colocar el efector final en un punto conocido del espacio, ¿qué valores deben tomar q1, q2 y q3?

ENTRADA: x, y, z → CINEMÁTICA INVERSA → SALIDA: q1, q2, q3

Un mismo punto cartesiano puede admitir más de una configuración articular. La rutina utilizada en este robot contempla dos ramas geométricas y permite seleccionar una solución mediante la variable use_elbow.

2. Robot 3R utilizado

Las rutinas trabajan con un robot 3R de longitudes l1 = 0,50 m, l2 = 0,30 m y l3 = 0,20 m. La convención angular empleada en todas las rutinas debe ser la misma:

- q1 gira alrededor del eje Z y determina la orientación del plano vertical del brazo.
- q2 se mide desde la dirección radial horizontal hacia el eslabón L2.
- q3 = 0 cuando L3 es una prolongación de L2.
- q3 positivo eleva L3 respecto de la prolongación de L2.

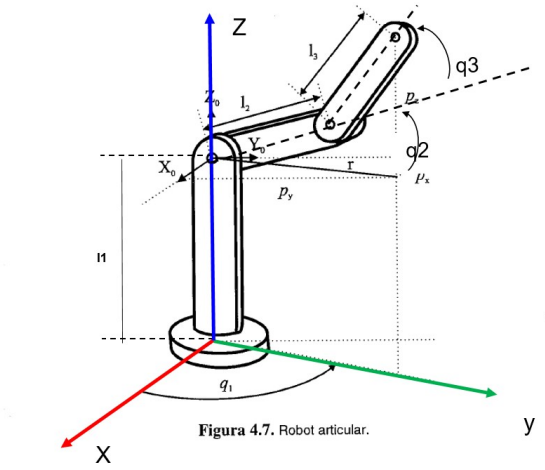


Figura 4.7. Robot articular.

Esta coherencia de convenciones es fundamental: la cinemática directa, la inversa y la función de dibujo deben interpretar los ángulos exactamente de la misma manera.

3. Rutina cinematica_directa_3R.m

Esta función calcula la posición del efector final a partir de una configuración articular conocida.

q1,q2,q3 [grados] → cinematica_directa_3R → x,y,z [m]

Cómo se utiliza

```
[x,y,z] = cinematica_directa_3R(q1,q2,q3)
```

Por ejemplo:

```
[x,y,z] = cinematica_directa_3R(0,30,30)
```

MATLAB devuelve tres valores: x, y y z. Esos valores representan las coordenadas del extremo del robot para la configuración articular introducida.

Qué calcula internamente

Primero calcula la distancia radial r del efector respecto del eje Z. Luego calcula su altura z y, finalmente, proyecta la distancia radial sobre los ejes X e Y mediante q1.

```
r = l2*cosd(q2) + l3*cosd(q2 + q3);
z = l1 + l2*sind(q2) + l3*sind(q2 + q3);
x = r*cosd(q1);
y = r*sind(q1);
```

4. Rutina cinematica_inversa_3R.m

Esta función recibe un punto cartesiano deseado y calcula los tres ángulos que permiten alcanzar ese punto.

x,y,z [m] → cinematica_inversa_3R → q1,q2,q3 [grados]

Cómo se utiliza

```
[q1,q2,q3] = cinematica_inversa_3R(x,y,z)
```

Ejemplo de prueba:

```
[q1,q2,q3] = cinematica_inversa_3R(0.3598,0,0.8232)
```

Con la rama 'up' seleccionada, este punto produce aproximadamente q1 = 0°, q2 = 30° y q3 = 30°. Las pequeñas diferencias que puedan aparecer se deben al redondeo de las coordenadas introducidas.

Controles que realiza

La función proyecta el punto en el plano vertical del brazo y verifica primero si el punto está dentro del alcance geométrico. Si no lo está, MATLAB interrumpe el cálculo e informa que el punto está fuera del alcance del robot.

Después aplica la ley del coseno para obtener las dos posibles soluciones de q3 y calcula el q2 correspondiente a cada una. En la versión actual se selecciona la solución mediante:

```
use_elbow = "up";
```

En esta rutina, 'up' corresponde a q3 positivo y 'down' a q3 negativo.

5. Rutina dibujar_robot_3R.m

Esta función no calcula una trayectoria ni busca una solución inversa. Su tarea es exclusivamente gráfica: recibe una configuración articular y representa el robot en esa pose.

q1,q2,q3 → dibujar_robot_3R → dibujo 3D de una pose

Cómo se utiliza

```
dibujar_robot_3R(0,30,30)
```

La función calcula los puntos O, A, B y C que corresponden a la base, las articulaciones y el efector final. Luego une esos puntos mediante plot3. El punto C debe coincidir con la posición obtenida por la cinemática directa para los mismos q1, q2 y q3.

Esto permite utilizar el dibujo como una verificación visual de las configuraciones articulares.

6. ¿Por qué generar una sucesión de Q?

Hasta aquí, una configuración Q = [q1 q2 q3] representa una sola **pose** del robot. Para observar movimiento necesitamos muchas poses intermedias entre una configuración inicial y una final.

Q inicial → Q2 → Q3 → ... → Q final

La matriz Q almacena esa sucesión. En los programas desarrollados, cada fila contiene una pose completa:

```
Q = [ q1(1)  q2(1)  q3(1)
      q1(2)  q2(2)  q3(2)
        ...    ...    ...
      q1(N)  q2(N)  q3(N) ];
```

Por lo tanto, Q(i,:) significa: configuración completa del robot en el instante o paso i. Todavía no se está definiendo una ley temporal física; se está construyendo una secuencia discreta de poses articulares.

7. Programa generacion_de_qs.m

Este script genera la matriz Q entre una configuración inicial y una configuración final. No dibuja el robot.

Q inicial + Q final + paso máximo → generacion_de_qs → matriz Q

Datos que solicita

1. q1 inicial, q2 inicial y q3 inicial.
2. q1 final, q2 final y q3 final.
3. Paso angular máximo deseado, expresado en grados.

Cómo sincroniza las articulaciones

Primero calcula cuánto debe desplazarse cada articulación. Después busca cuál de las tres debe recorrer el mayor ángulo. Esa articulación determina la cantidad de intervalos.

```
Dq1 = q1_final - q1_inicial;
Dq2 = q2_final - q2_inicial;
Dq3 = q3_final - q3_inicial;
Dq_max = max(abs([Dq1 Dq2 Dq3]));
N_intervalos = ceil(Dq_max / paso);
```

Una vez conocida la cantidad común de intervalos, cada articulación recibe su propio incremento. Así, las tres comienzan juntas y alcanzan su valor final en la misma cantidad de pasos.

```
paso_q1 = Dq1 / N_intervalos;
paso_q2 = Dq2 / N_intervalos;
paso_q3 = Dq3 / N_intervalos;
```

El resultado final es la matriz Q. Este programa es especialmente útil para estudiar primero la generación de configuraciones sin mezclarla todavía con la representación gráfica.

8. Programa genera_qs_y_dibuja.m

Este script toma la idea anterior y agrega la animación. Primero genera cada fila de Q y, dentro del mismo ciclo for, envía esa configuración a dibujar_robot_3R.

Datos articulares → genera Q(i,:) → dibujar_robot_3R(Q(i,:)) → movimiento

En cada iteración i se calculan los tres ángulos de la nueva pose:

```
Q(i,1) = q1_inicial + (i-1)*paso_q1;
Q(i,2) = q2_inicial + (i-1)*paso_q2;
Q(i,3) = q3_inicial + (i-1)*paso_q3;
```

Inmediatamente después se dibuja esa fila:

```
dibujar_robot_3R(Q(i,1), Q(i,2), Q(i,3));
```

La pausa de 0,1 s permite observar visualmente el cambio entre poses. El programa también contempla el caso especial en el que la configuración inicial y la final son iguales: en ese caso genera una única pose y simplemente la dibuja.

9. Diferencia entre las cinco rutinas

Rutina	Entrada	Salida / resultado	Función principal
cinematica_directa_3R	q1,q2,q3	x,y,z	Localizar el efector
cinematica_inversa_3R	x,y,z	q1,q2,q3	Encontrar los ángulos
dibujar_robot_3R	q1,q2,q3	Gráfico 3D	Dibujar una pose
generacion_de_qs	Q inicial, Q final, paso	Matriz Q	Generar poses intermedias
genera_qs_y_dibuja	Q inicial, Q final, paso	Matriz Q + animación	Generar y visualizar movimiento

10. Esquema general de trabajo

Configuración articular
q1,q2,q3
↓

Cinemática directa → x,y,z

Punto deseado x,y,z
↓
Cinemática inversa → q1,q2,q3

Q inicial + Q final
↓
Generación de Q → sucesión de poses
↓
dibujar_robot_3R → animación

11. Pruebas recomendadas para el alumno

Una buena forma de comprobar que las rutinas son coherentes es realizar pruebas encadenadas.

Prueba A: directa → inversa

```
[x,y,z] = cinematica_directa_3R(0,30,30)
[q1,q2,q3] = cinematica_inversa_3R(x,y,z)
```

Si se utiliza la misma rama geométrica, la inversa debe recuperar aproximadamente los ángulos originales.

Prueba B: cálculo → dibujo

```
[x,y,z] = cinematica_directa_3R(0,30,30);
dibujar_robot_3R(0,30,30)
```

El extremo C del dibujo debe corresponder a las coordenadas x,y,z calculadas por la cinemática directa.

Prueba C: movimiento articular

Ejecutar genera_qs_y_dibuja, introducir una configuración inicial, una final y un paso angular. Al terminar, observar la matriz Q: la primera fila debe coincidir con la configuración inicial y la última con la configuración final.

12. Idea que prepara el paso siguiente

Hasta este punto el movimiento se genera en el espacio articular: elegimos valores iniciales y finales de q1, q2 y q3 y construimos las configuraciones intermedias. El paso siguiente consiste en definir una trayectoria en el espacio cartesiano, por ejemplo una recta formada por puntos (x,y,z), calcular la cinemática inversa para cada punto y obtener así la matriz Q que hará que el efector final siga esa trayectoria.

Puntos de una recta x,y,z → cinemática inversa punto a punto → matriz Q → dibujo del robot

Fin de la guía

De la recta 2D a una trayectoria cartesiana para un robot 3R

Representación paramétrica en MATLAB y conexión con la cinemática inversa

El objetivo de este apunte es construir, paso a paso, una idea muy importante en robótica: partir de la ecuación de una recta, representarla mediante un parámetro, generar una sucesión de puntos y utilizar esos puntos como posiciones deseadas del efector final de un robot.

Recta 2D → forma paramétrica → recta 3D → segmento P1-P2 → puntos x,y,z → cinemática inversa → movimiento

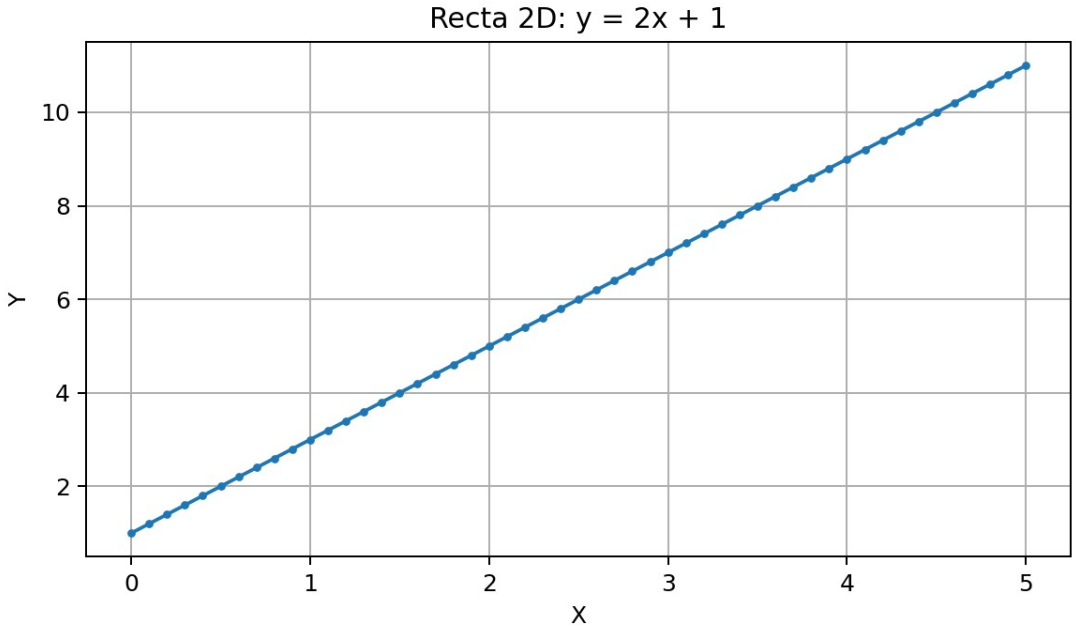
1. La recta en dos dimensiones

En el plano XY, una forma conocida de escribir una recta es la ecuación explícita:

$$y = m\,x + b$$

donde m es la pendiente de la recta y b es la ordenada al origen. Por ejemplo, el programa recta_2_D.m utiliza m = 2 y b = 1, por lo que la recta es:

$$y = 2x + 1$$



2. ¿Por qué pasar a una forma paramétrica?

Para mover un robot no alcanza con conocer una ecuación geométrica: necesitamos generar puntos concretos que el efector pueda recorrer. La parametrización introduce una variable t que permite obtener

esos puntos de manera ordenada. En este caso necesitamos puntos en X y en Y que dependan de otra variable.

En el ejemplo 2D podemos elegir x = t. Si sustituimos x por t en y = 2x + 1 obtenemos:

$$x(t) = t \qquad y(t) = 2t + 1$$

Cada valor de t genera un punto distinto (x,y) de la recta. Si t toma muchos valores, MATLAB genera una sucesión de puntos que, al representarse, forman la recta.

```
m = 2;
b = 1;

t = 0:0.1:5;

x = t;
y = m*t + b;

plot(x,y,'o-');
```

Por ejemplo, t = 0 produce el punto (0,1), t = 1 produce (1,3) y t = 2 produce (2,5). El parámetro t actúa entonces como un índice continuo que recorre la geometría.

3. Extensión de la idea a tres dimensiones

En el espacio ya no alcanza con dos coordenadas. Cada punto debe tener tres componentes: x, y y z. Una recta 3D puede expresarse paramétricamente haciendo que las tres coordenadas dependan del mismo parámetro t.

El programa recta_3_D.m utiliza x como parámetro y define:

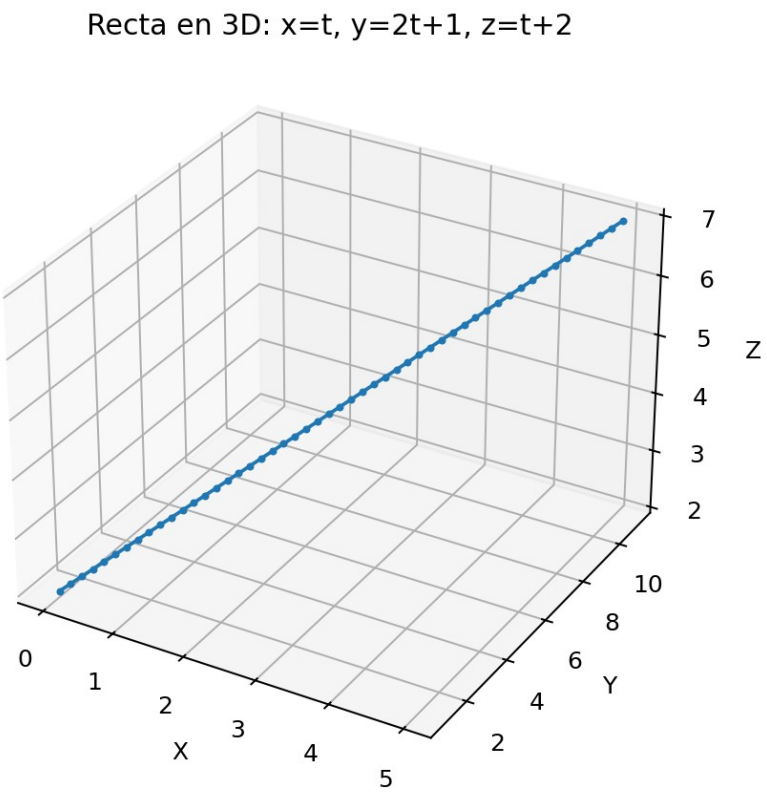
$$x(t) = t \qquad y(t) = 2t + 1 \qquad z(t) = t + 2$$

La idea es exactamente la misma que en 2D. La diferencia es que ahora cada valor del parámetro genera una terna (x,y,z), es decir, un punto del espacio.

```
x = 0:0.1:5;

y = 2*x + 1;
z = x + 2;

plot3(x,y,z,'o-');
```



4. Recta definida por dos puntos P1 y P2

Para robótica resulta especialmente útil definir un segmento en el espacio de tareas (cartesiano) mediante un punto inicial P1 y un punto final P2. Así podemos indicar desde dónde hasta dónde queremos que se desplace el efector final. Supongamos que el efector final de nuestro robot queremos que se desplace en línea recta desde P1 a P2 :

```
P1 = [0.20  0.20  0.60];
P2 = [0.35  0.25  0.70];
```

La forma vectorial paramétrica del segmento es:

$$\mathbf{P(t) = P1 + t\ (P2 - P1) \qquad con\ 0 \leq t \leq 1}$$

Esta expresión tiene una interpretación muy sencilla. Cuando t = 0 se obtiene exactamente P1. Cuando t = 1 se obtiene exactamente P2. Para valores entre 0 y 1 aparecen todos los puntos intermedios.

Para los puntos utilizados en el programa:

```
P2 - P1 = [0.15  0.05  0.10]
```

Por lo tanto, las tres ecuaciones paramétricas son:

$$\mathbf{x = 0.20 + 0.15\ t}$$
$$\mathbf{y = 0.20 + 0.05\ t}$$
$$\mathbf{z = 0.60 + 0.10\ t}$$

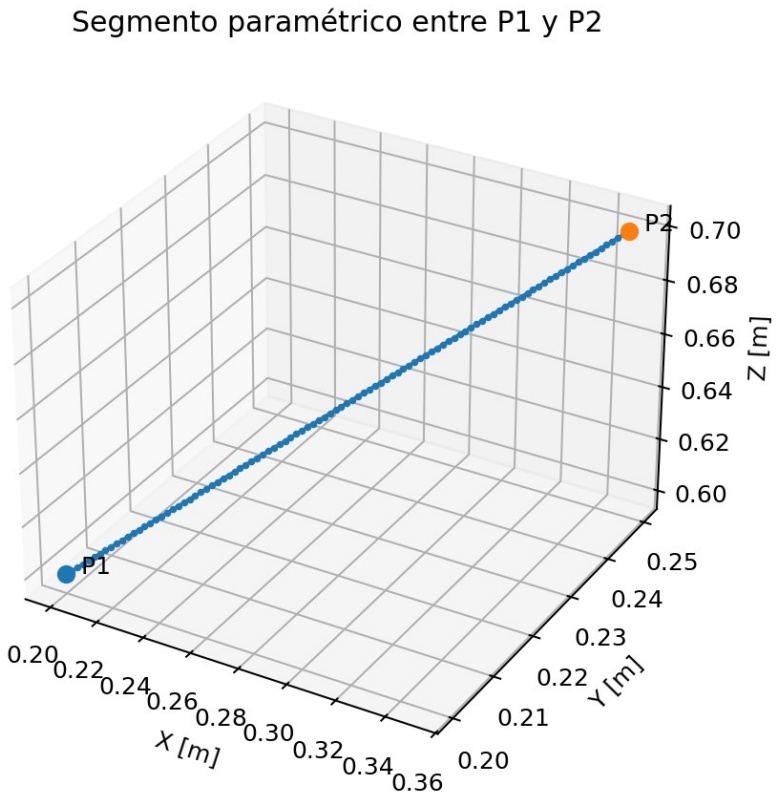
Acá se ve claramente que para t= 0 la recta comienza en **x = 0.20 y = 0.20 y z = 0.60** y para t = 1 entoces X= `0.35, Y=  0.25 y Z  0.70` que es justo P2

El script recta_3_d_ini_fin utiliza t = 0:0.01:1. Esto genera 101 valores de t y, por lo tanto, 101 puntos cartesianos desde P1 hasta P2, incluyendo ambos extremos.

```
t = 0:0.01:1;

x = 0.20 + 0.15*t;
y = 0.20 + 0.05*t;
z = 0.60 + 0.10*t;

plot3(x,y,z,'o-');
```



5. ¿Qué queda guardado en MATLAB?

Después de ejecutar la parametrización, x, y y z y de usar el programa : recta_3_d_ini_fin.m son vectores del mismo tamaño. Los elementos con el mismo índice forman un punto cartesiano:

Punto i = [x(i), y(i), z(i)]

Por ejemplo, x(1), y(1), z(1) corresponden a P1. Los elementos siguientes corresponden a puntos intermedios y el último conjunto corresponde a P2.

6. De los puntos cartesianos al movimiento del robot

Aquí aparece la conexión con la cinemática inversa. Para cada punto de la recta se toman sus tres coordenadas y se solicita qué valores q1, q2 y q3 necesita el robot para colocar allí su efector final.

x(i),y(i),z(i) → cinematica_inversa_3R → q1,q2,q3 → dibujar_robot_3R

El programa **genera_recta_3D.m** implementa exactamente este procedimiento aclarando que los puntos de toda la recta : **x(i),y(i),z(i) están cargados ahora mediante un ciclo for calculamos primero la cinemática inversa, con ella obtenemos todas las poses correspondientes al robot o sea todos los q1(i),q2(i),q3(i) que correspondan y luego con todos los valores de las articulaciones corremos la función** `dibujar_robot_3R(q1,q2,q3);` y veremos el robot en movimiento

```
for i = 1:length(x)

    px = x(i);
    py = y(i);
    pz = z(i);

    [q1,q2,q3] = cinematica_inversa_3R(px,py,pz);

    dibujar_robot_3R(q1,q2,q3);

    hold on;
    plot3(x(1:i), y(1:i), z(1:i), 'o');

    drawnow;

end
```

En cada vuelta del ciclo se selecciona un nuevo punto cartesiano. La cinemática inversa transforma ese punto en una configuración articular. Luego la función de dibujo representa el robot en esa configuración. Al repetir el proceso para todos los puntos, el efector final del robot debiera recorrer el segmento de P1 a P2.

7. El flujo completo

1. Elegir P1 y P2

↓

2. Crear t entre 0 y 1

↓

3. Generar x(t), y(t), z(t)

↓

4. Tomar cada punto [x(i), y(i), z(i)]

↓

5. Calcular [q1,q2,q3] con cinemática inversa

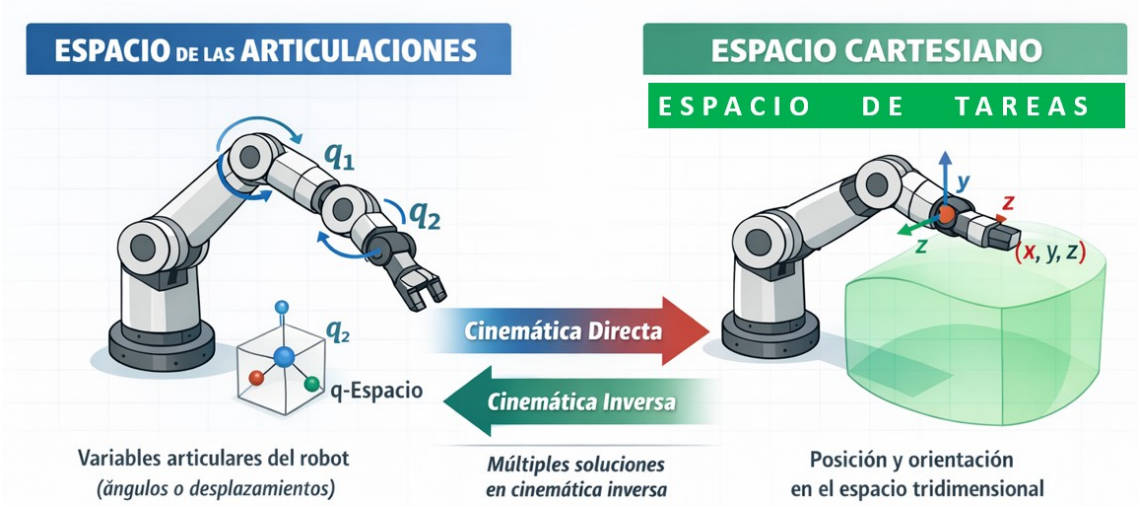
↓

6. Dibujar el robot en esa configuración

↓

7. Repetir hasta llegar a P2

8. Diferencia importante: espacio cartesiano y espacio articular



En los programas anteriores de generación de Q, se elegían directamente una configuración articular inicial y una final, y se generaban valores intermedios de q1, q2 y q3. En este nuevo procedimiento la trayectoria se define primero en el espacio cartesiano o de tareas (X,Y,Z). para luego calcular los q1,q2,q3 que correspondan a cada punto en el espacio de tareas y con esos q1,q2,q3, dibujar al robot en cada pose.

Procedimiento	Se define primero	Se calcula después
Generación de Q	q1, q2, q3	Posición resultante del efector
Recta cartesiana	x, y, z	q1, q2, q3 mediante cinemática inversa

Esta diferencia es esencial: si el objetivo es que el efector final siga una recta en el espacio, conviene generar primero los puntos de esa recta y resolver la cinemática inversa punto por punto.

9. Resumen conceptual

La parametrización transforma una descripción geométrica en una sucesión ordenada de puntos. Esa sucesión es precisamente lo que necesitamos para conectar la geometría de una trayectoria con la cinemática del robot.

Geometría de la recta → puntos cartesianos → cinemática inversa → configuraciones articulares → movimiento

De la ecuación del círculo a la trayectoria del efector final

Apunte introductorio para Robótica

Objetivo: partir de la ecuación elemental de un círculo, reconocer su forma paramétrica con seno y coseno, extenderla a tres dimensiones y comprender por qué esta representación es conveniente para definir la trayectoria cartesiana de un efector final.

1. El círculo conocido: plano XY y centro en el origen

Comencemos con el caso más familiar: un círculo de radio R, ubicado en el plano XY y con centro en el origen del sistema de coordenadas, O = (0,0).

$$x^2 + y^2 = R^2$$

Esta ecuación expresa que todo punto P = (x,y) del círculo se encuentra a una distancia R del origen. Es una ecuación muy útil para describir geométricamente el círculo.

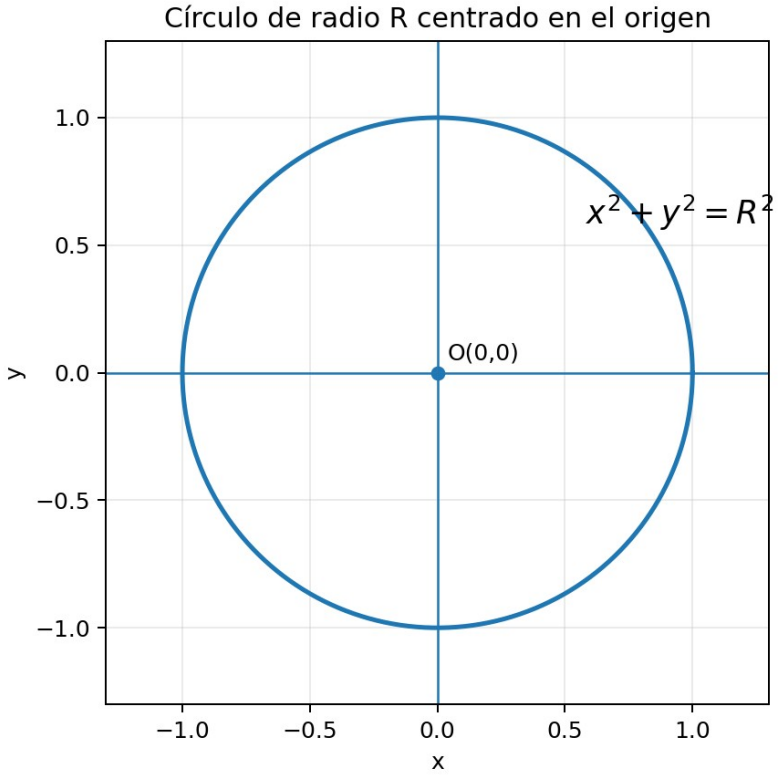


Figura 1. Círculo de radio R centrado en el origen del plano XY.

2. Introducimos un ángulo para recorrer el círculo

Ahora elegimos un punto P del círculo y trazamos el radio que une el origen con ese punto. Llamemos w al ángulo que dicho radio forma con el eje X.

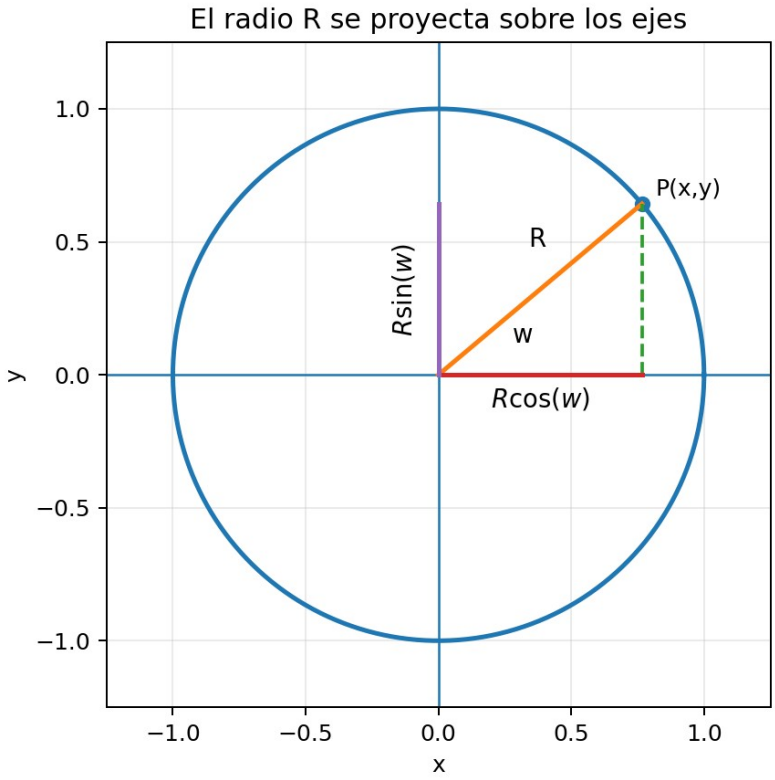


Figura 2. Proyecciones del radio R sobre los ejes X e Y.

Por trigonometría, las proyecciones del radio sobre los ejes son:

$$\cos(w) = x / R$$

$$\sin(w) = y / R$$

Despejando x e y obtenemos:

$$\mathbf{x = R\ \cos(w)}$$

$$\mathbf{y = R\ \sin(w)}$$

Ésta es la forma paramétrica del círculo. No describe otro círculo: describe exactamente el mismo círculo, pero ahora las coordenadas x e y se obtienen a partir de un parámetro, el ángulo w.

3. Comprobación: es el mismo círculo

Si sustituimos x = R cos(w) e y = R sin(w) en la ecuación original:

$$\mathbf{x^2 + y^2 = R^2}$$

$$(R\ \cos(w))^2 + (R\ \sin(w))^2 = R^2$$

$$R^2\ [\cos^2(w) + \sin^2(w)] = R^2$$

$$\text{como } \cos^2(w) + \sin^2(w) = 1$$

$$\mathbf{R^2 = R^2}$$

Por lo tanto, la ecuación cartesiana y la forma con seno y coseno representan la misma geometría.

4. ¿Por qué conviene usar seno y coseno?

La ecuación $x^2 + y^2 = R^2$ nos permite saber si un punto pertenece al círculo, **pero no nos entrega directamente una secuencia ordenada de puntos para recorrerlo.**

En cambio, si hacemos variar w (o un theta) desde 0° hasta 360°, las ecuaciones x = R cos(w) e y = R sin(w) generan automáticamente los puntos del círculo en el orden en que deben recorrerse.

w	x	y
0°	R	0
90°	0	R
180°	−R	0
270°	0	−R
360°	R	0

Así, el parámetro w funciona como una especie de “cursor” que va indicando qué punto del círculo queremos obtener.

5. Paso a tres dimensiones

En el espacio tridimensional cada punto tiene tres coordenadas: P = (x,y,z). Un círculo, sin embargo, sigue siendo una figura plana. Para ubicarlo en 3D debemos indicar en qué plano se encuentra.

Si el círculo está en el plano XY:

$$x = R\cos(w)$$

$$y = R\sin(w)$$

$$z = 0$$

Si queremos un círculo vertical en el plano XZ, entonces la coordenada y permanece constante. Para un círculo centrado en el origen:

$$x = R\cos(w)$$

$$y = 0$$

$$z = R\sin(w)$$

Figura 3. El mismo concepto de círculo, ahora ubicado en el plano XZ del espacio tridimensional.

6. Trasladar el círculo a otra posición del espacio

Hasta aquí el círculo estaba centrado en el origen. En el programa MATLAB utilizado con nuestro robot 3R, el círculo se traslada a un punto del espacio que se encuentra dentro del volumen de trabajo del robot. Se utiliza:

$$R = 0.10 \text{ m}$$

$$C = (x_c, y_c, z_c) = (0.30, 0.10, 0.65) \text{ m}$$

$$x = 0.30 + 0.10 \cos(w) \qquad y = 0.10 \qquad z = 0.65 + 0.10 \sin(w)$$

Estas ecuaciones describen un círculo vertical de radio 0.10 m, contenido en un plano paralelo a XZ. La coordenada y permanece constante en 0.10 m. El círculo tiene un diámetro de 0.20 m y su centro está ubicado en C = (0.30, 0.10, 0.65) m.

7. Relación con la trayectoria de un robot

Para un robot, lo importante no es solamente conocer la forma geométrica del círculo, sino disponer de una sucesión ordenada de posiciones que el efector final deberá alcanzar.

Cada valor de w genera una posición cartesiana deseada del efector final:

$$P(w) = [\ x_c + R \cos(w),\ \ y_c,\ \ z_c + R \sin(w)\ ]$$

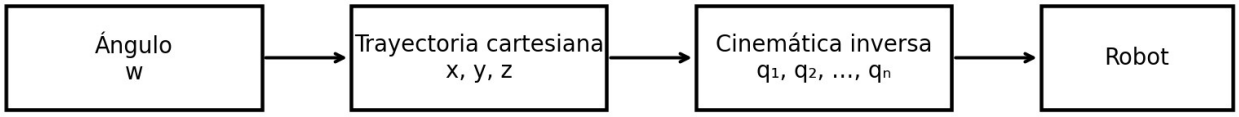


Figura 4. La trayectoria cartesiana se transforma luego en variables articulares mediante cinemática inversa.

La cinemática inversa toma cada posición cartesiana deseada (x,y,z) y calcula los valores articulares q1, q2 y q3 que debe adoptar nuestro robot 3R para que su efector final llegue a ese punto.

8. Correspondencia con MATLAB

```
R = 0.10;          % Radio [m]

x_c = 0.30;        % Centro en X [m]
y_c = 0.10;        % Centro en Y [m]
z_c = 0.65;        % Centro en Z [m]

paso = 5;          % Paso angular [grados]
w = 0:paso:360;

x = x_c + R*cosd(w);
y = y_c + zeros(size(w));
z = z_c + R*sind(w);

P = [x' y' z'];

plot3(x,y,z,'o-','LineWidth',1.5);

hold on;
grid on;

plot3(x_c,y_c,z_c,'o', ...
      'MarkerSize',8, ...
      'MarkerFaceColor','k');

xlabel('X [m]');
ylabel('Y [m]');
zlabel('Z [m]');

title('Circulo en el plano XZ');
```

Al terminar el programa quedan cargados los vectores x, y y z y también la matriz P. En este programa P se construye como P = [x' y' z']; por lo tanto, cada FILA de P representa un punto cartesiano del círculo: P(i,:) = [x(i) y(i) z(i)]. Con paso = 5°, w toma valores desde 0° hasta 360° y se generan 73 puntos; el primero y el último coinciden para cerrar el círculo.

9. Idea fundamental para recordar

La ecuación cartesiana **$x^2 + y^2 = R^2$** describe qué puntos forman el círculo.

La forma paramétrica **x = R cos(w), y = R sin(w)** permite generar esos puntos en orden.

En robótica, esta forma paramétrica permite generar en MATLAB una sucesión ordenada de puntos cartesianos. En nuestro programa esos puntos quedan almacenados por filas en P y luego pueden recorrerse uno por uno. Para cada fila P(i,:), la cinemática inversa calcula q1, q2 y q3 y el robot adopta sucesivamente esas configuraciones, haciendo que su efector final dibuje el círculo.

Generación de un cuadrado y animación del robot 3R

Guía simple y didáctica de los programas utilizados

Robot utilizado: 3 articulaciones rotativas (3R), con longitudes l1 = 0,50 m, l2 = 0,30 m y l3 = 0,20 m. Los ángulos Q1, Q2 y Q3 se trabajan en grados.

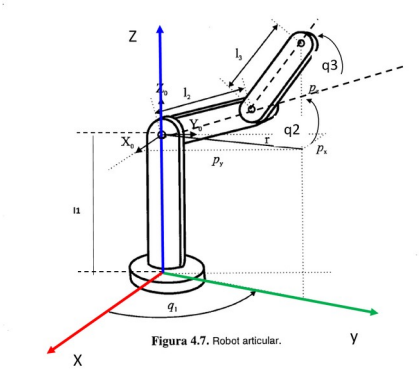


Figura 4.7. Robot articular.

1. Idea general

El objetivo es hacer que el extremo del robot recorra un cuadrado. Primero se construyen los puntos del cuadrado en coordenadas cartesianas. Después se calcula, para cada punto, qué valores deben tener Q1, Q2 y Q3. Finalmente se van enviando esas configuraciones al programa que dibuja el robot.

CUADRADO → P → CINEMÁTICA INVERSA NUMÉRICA → Q → ANIMACIÓN

P es la matriz de puntos cartesianos [x y z]. Q es la matriz de configuraciones articulares [Q1 Q2 Q3]. Cada fila de P se corresponde con la misma fila de Q.

2. Programas y rutinas que intervienen

Programa / rutina	Función
genera_cuadrado_3R.m	Genera los puntos cartesianos del cuadrado y los guarda en P.
calcula_qs_cuadrado.m	Recorre P y calcula Q1, Q2 y Q3 para cada punto.
cinematica_inversa_3R_numerica.m	Busca numéricamente Q1, Q2 y Q3 para alcanzar un punto Pd.
cinematica_directa_3R_DH.m	Calcula la posición actual del extremo a partir de Q1, Q2 y Q3.
matrizDenavitHartenberg.m	Construye las matrices homogéneas DH utilizadas por la cinemática directa.
jacobiano_3R.m	Calcula el Jacobiano de posición del robot 3R.
correccion_DLS_3R.m	Calcula la corrección articular mediante Damped Least Squares.
anima_cuadrado_3R.m	Lee sucesivamente las filas de Q y ordena dibujar cada postura.
dibujar_robot_3R.m	Dibuja y actualiza gráficamente el mismo robot 3R sin parpadeo.

3. Programa: genera_cuadrado_3R.m

Este es el punto de partida. Define un cuadrado dentro del espacio de trabajo del robot. El cuadrado se divide en varios puntos y esos puntos se guardan fila por fila en la matriz P.

```
% Ejemplo conceptual de la matriz obtenida:
P = [x1  y1  z1
     x2  y2  z2
     x3  y3  z3
     ...
     xn  yn  zn];
```

En la versión que estamos usando se toman N = 10 puntos por lado, evitando repetir los vértices al unir los lados. El resultado es una matriz P de 37 filas por 3 columnas.

```
P =

    0.1500   -0.1000    0.6500
    0.1722   -0.1000    0.6500
    0.1944   -0.1000    0.6500
    0.2167   -0.1000    0.6500
    0.2389   -0.1000    0.6500
    0.2611   -0.1000    0.6500
    0.2833   -0.1000    0.6500
    0.3056   -0.1000    0.6500
    0.3278   -0.1000    0.6500
    0.3500   -0.1000    0.6500
```

Prueba: ejecutar en MATLAB:

```
genera_cuadrado_3R
```

Se debe observar el cuadrado en una figura 3D y debe aparecer la variable P en el Workspace. Puede comprobarse con:

```
size(P)
```

El resultado esperado es 37 × 3.

4. Programa: calcula_qs_cuadrado.m

Este programa toma los puntos de P uno por uno. Para cada punto forma el vector deseado Pd y llama a la cinemática inversa numérica.

```
Pd = P(i,:)';

[q,success,iter] = ...
    cinematica_inversa_3R_numerica(Pd,q0);

Q(i,:) = q';
```

La salida q contiene los tres ángulos calculados: Q1, Q2 y Q3. Cada solución se guarda como una fila de la matriz Q.

$$Q = \begin{bmatrix} q_{1,1} & q_{2,1} & q_{3,1} \\ q_{1,2} & q_{2,2} & q_{3,2} \\ q_{1,3} & q_{2,3} & q_{3,3} \\ \vdots & \vdots & \vdots \end{bmatrix}$$

P(i,:) ↔ Q(i,:)

5. ¿Por qué se usa q0 = q?

Para el primer punto se entrega una postura inicial q0. Cuando el algoritmo encuentra una solución q, esa solución se utiliza como semilla del punto siguiente:

```
q0 = q;
```

Como los puntos consecutivos del cuadrado están próximos entre sí, también es razonable que las configuraciones articulares consecutivas estén próximas. Esta elección ayuda a mantener continuidad en la solución numérica.

6. Rutinas internas de la cinemática inversa

calcula_qs_cuadrado.m no resuelve por sí solo la cinemática inversa. Llama a cinematica_inversa_3R_numerica.m. Dentro de esa rutina se repite el siguiente proceso:

1. cinematica_directa_3R_DH.m calcula dónde está actualmente el extremo del robot.

2. Se calcula el error cartesiano e = Pd - Pactual.

3. jacobiano_3R.m calcula el Jacobiano de posición.

4. correccion_DLS_3R.m obtiene una pequeña corrección de Q1, Q2 y Q3.

5. Se actualizan los ángulos y se repite hasta que el error sea suficientemente pequeño.

La cinemática directa utiliza matrizDenavitHartenberg.m para construir las transformaciones del mismo robot 3R.

7. Resultado: la matriz Q

Si todos los puntos convergen correctamente, al terminar calcula_qs_cuadrado.m tendremos:

```
Q = [Q1_1  Q2_1  Q3_1
     Q1_2  Q2_2  Q3_2
     Q1_3  Q2_3  Q3_3
     ...
     Q1_n  Q2_n  Q3_n];
```

Por ejemplo, Q(5,:) contiene los tres ángulos que hacen que el extremo del robot llegue al punto cartesiano P(5,:).

```
Matriz Q obtenida:
  -33.6898   -1.9749  128.6795
  -30.1432   -4.4424  124.4273
  -27.2164   -6.4188  119.8326
  -24.7752   -7.9035  114.9191
  -22.7144   -8.9122  109.6931
  -20.9558   -9.4661  104.1458
  -19.4400   -9.5847   98.2546
  -18.1219   -9.2811   91.9784
  -16.9661   -8.5569   85.2527
  -15.9454   -7.3979   77.9791
  -12.5288   -7.7349   79.8938
```

8. Programa: anima_cuadrado_3R.m

Una vez que P y Q están calculadas, podemos realizar una primera animación. El programa recorre las filas de Q y toma los tres ángulos de cada configuración.

```
figure

for i = 1:size(Q,1)

    q1 = Q(i,1);
    q2 = Q(i,2);
    q3 = Q(i,3);

    dibujar_robot_3R(q1,q2,q3);

    hold on

    plot3(P(1:i,1), ...
          P(1:i,2), ...
          P(1:i,3), ...
          'o-');

    drawnow
    pause(0.1)
```

```
end
```

En cada vuelta del for se toma una fila diferente de Q. La rutina dibujar_robot_3R.m actualiza la postura del robot y plot3 deja visible la parte del cuadrado que se ha recorrido.

9. Rutina: dibujar_robot_3R.m

Es la misma rutina de dibujo que venimos utilizando para este robot. Trabaja con l1 = 0,50 m, l2 = 0,30 m y l3 = 0,20 m y recibe Q1, Q2 y Q3 en grados.

Su geometría coincide con la del robot utilizado en la cinemática directa, el Jacobiano y la cinemática inversa numérica. Calcula los puntos de las articulaciones y del extremo, y actualiza los objetos gráficos sin borrar toda la figura en cada paso.

10. Cómo probar todo el conjunto

Conviene probar el sistema en tres etapas, manteniendo las variables P y Q en el Workspace:

Primero: ejecutar genera_cuadrado_3R.m. Verificar que aparece el cuadrado y que existe P.

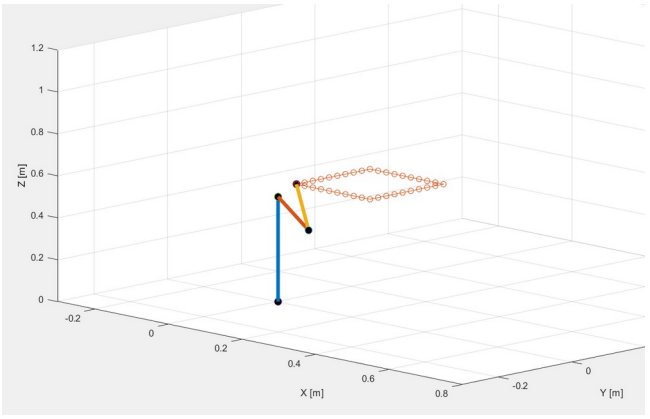
Segundo: ejecutar calcula_qs_cuadrado.m. Verificar que todos los puntos convergen y que existe Q.

Tercero: ejecutar anima_cuadrado_3R.m. El robot debe adoptar sucesivamente las configuraciones de Q y recorrer el cuadrado.

```
genera_cuadrado_3R
calcula_qs_cuadrado
anima_cuadrado_3R
```

11. Esquema completo de llamadas

```
genera_cuadrado_3R.m
        |
        v
        P
        |
        v
calcula_qs_cuadrado.m
        |
        +--> cinematica_inversa_3R_numerica.m
                  |
                  +--> cinematica_directa_3R_DH.m
                  |       |
                  |       +-->
matrizDenavitHartenberg.m
                  |
                  +--> jacobiano_3R.m
                  |
                  +--> correccion_DLS_3R.m
        |
        v
        Q
        |
        v
anima_cuadrado_3R.m
        |
        +--> dibujar_robot_3R.m
```



12. Qué hemos conseguido y qué falta

Con estos programas ya podemos definir una figura cartesiana, calcular automáticamente las configuraciones articulares necesarias y visualizar al robot pasando por ellas.

FIGURA → P → Q → ANIMACIÓN

Esta animación todavía es una sucesión de configuraciones calculadas para los puntos del cuadrado. El paso siguiente será introducir una generación de trayectoria articular con tiempo de muestreo y velocidad, obteniendo puntos intermedios entre configuraciones. Ese será el paso hacia una estructura del tipo q_total y Dq_total.

J = jacobiano_3R(q1,q2,q3)

Jacobiano simbólico del robot 3R

Jacobiano de posición para la cinemática inversa numérica

1. Cinemática directa de partida

Para el robot 3R utilizamos:

$$x = [l2 \cos(q2) + l3 \cos(q2 + q3)] \cos(q1)$$

$$y = [l2 \cos(q2) + l3 \cos(q2 + q3)] \sin(q1)$$

$$z = l1 + l2 \sin(q2) + l3 \sin(q2 + q3)$$

2. Definición del Jacobiano

El Jacobiano de posición se obtiene derivando las tres coordenadas cartesianas x, y, z respecto de las tres variables articulares q1, q2, q3:

$$J = \begin{bmatrix} \partial x/\partial q1 & \partial x/\partial q2 & \partial x/\partial q3 \\ \partial y/\partial q1 & \partial y/\partial q2 & \partial y/\partial q3 \\ \partial z/\partial q1 & \partial z/\partial q2 & \partial z/\partial q3 \end{bmatrix}$$

3. Jacobiano simbólico desarrollado

$$J = \begin{bmatrix} -[l2 \cos(q2)+l3 \cos(q2+q3)] \sin(q1) & -[l2 \sin(q2)+l3 \sin(q2+q3)] \cos(q1) & -l3 \sin(q2+q3) \cos(q1) \\ [l2 \cos(q2)+l3 \cos(q2+q3)] \cos(q1) & -[l2 \sin(q2)+l3 \sin(q2+q3)] \sin(q1) & -l3 \sin(q2+q3) \sin(q1) \\ 0 & l2 \cos(q2)+l3 \cos(q2+q3) & l3 \cos(q2+q3) \end{bmatrix}$$

4. Forma abreviada

Para facilitar la lectura definimos:

$$A = l2 \cos(q2) + l3 \cos(q2 + q3)$$

$$B = l2 \sin(q2) + l3 \sin(q2 + q3)$$

$$J = \begin{bmatrix} -A \sin(q1) & -B \cos(q1) & -l3 \sin(q2+q3) \cos(q1) \\ A \cos(q1) & -B \sin(q1) & -l3 \sin(q2+q3) \sin(q1) \\ 0 & A & l3 \cos(q2+q3) \end{bmatrix}$$

5. ¿Por qué no aparece l1?

l1 es una longitud constante que solamente desplaza verticalmente el robot. Al derivar x, y y z respecto de q1, q2 y q3, la derivada de l1 es cero. Por esa razón l1 no aparece en el Jacobiano.

6. Relación entre el Jacobiano y el movimiento

El Jacobiano relaciona pequeños cambios de las articulaciones con pequeños cambios de la posición cartesiana del efector final:

$$\Delta P \approx J\,\Delta q$$

Cada columna indica cómo se desplaza aproximadamente el efector final cuando se modifica una sola articulación.

7. Radianes y grados

El Jacobiano simbólico anterior corresponde a la derivación matemática estándar respecto de ángulos expresados en radianes. Como nuestros programas MATLAB utilizan q1, q2 y q3 en grados, mediante sind y cosd, utilizamos:

$$J_{grados} = (\pi/180)\,J_{radianes}$$

De esta manera, si Δq está expresado en grados, J_grados permite obtener directamente el correspondiente desplazamiento cartesiano aproximado en metros.

8. Uso en la cinemática inversa numérica

Dentro del método iterativo que estamos construyendo, primero calculamos el error cartesiano e = Pd - Pactual. Luego evaluamos el Jacobiano en la postura actual y utilizamos Damped Least Squares (DLS) (método de mínimos cuadrados amortiguados) para calcular la corrección articular Δq.

$$Pd \rightarrow CD \rightarrow e \rightarrow J \rightarrow DLS \rightarrow \Delta q \rightarrow q\ nueva$$

9. Rutina MATLAB asociada

```
J = jacobiano_3R(q1,q2,q3)
```

Entradas: q1, q2 y q3 en grados. Salida: matriz Jacobiana de posición J de tamaño 3×3, preparada para relacionar incrementos articulares en grados con desplazamientos cartesianos.

prueba_cinematica_inversa_3R.m
Programa de prueba de la Cinemática Inversa Numérica – Robot 3R

Explicación usando los nombres reales de los archivos MATLAB utilizados

1. Archivos que forman el conjunto

El ensayo completo está organizado alrededor de un programa principal de prueba y cinco rutinas. Los nombres utilizados a continuación son los que aparecen en los archivos suministrados.

Archivo / rutina	Tipo	Función
prueba_cinematica_inversa_3R.m	Script principal	Organiza la prueba completa y muestra los resultados.
cinematica_inversa_3R_numerica.m	Función principal de CI	Busca iterativamente Q1, Q2 y Q3 para alcanzar Pd.
cinematica_directa_3R_DH.m	Función auxiliar	Calcula x, y, z a partir de Q1, Q2 y Q3 mediante DH.
matrizDenavitHartenberg.m	Función auxiliar	Construye una matriz homogénea DH de una articulación.
jacobiano_3R.m	Función auxiliar	Calcula el Jacobiano de posición 3×3, trabajando con incrementos angulares en grados.
correccion_DLS_3R.m	Función auxiliar	Calcula ΔQ a partir de J, e y lambda mediante Damped Least Squares.

2. ¿Qué prueba exactamente **prueba_cinematica_inversa_3R.m**?

El script prueba todo el conjunto sin utilizar todavía una trayectoria. Primero genera un punto cartesiano alcanzable a partir de una postura conocida. Después parte de otra postura, llamada semilla, y le pide a la cinemática inversa numérica que encuentre una configuración articular que llegue al mismo punto.

Q conocido → cinematica_directa_3R_DH → Pd → cinematica_inversa_3R_numerica → Q solución → verificación

3. Primera parte: postura conocida

El programa prueba comienza fijando una postura conocida del robot:

```
q1_real = 30;
q2_real = 30;
q3_real = 30;
```

Estos valores no se entregan directamente a la cinemática inversa. Se usan solamente para fabricar un punto Pd cuya alcanzabilidad está garantizada.

En este caso Pd es :

Punto cartesiano deseado:

X= 0.3116

Y= 0.1799

Z= 0.8232

4. Generación del punto deseado Pd

El script llama exactamente a:

```
[xd,yd,zd] = cinematica_directa_3R_DH(q1_real,q2_real,q3_real);
```

La función cinematica_directa_3R_DH.m recibe Q1, Q2 y Q3 en grados y devuelve x, y, z en metros. Internamente utiliza las longitudes l1=0.50 m, l2=0.30 m y l3=0.20 m, construye A01, A12 y A23 y calcula T03=A01*A12*A23.

Para construir cada matriz DH llama a matrizDenavitHartenberg.m:

```
A01 = matrizDenavitHartenberg(q1, l1, 0, 90);
A12 = matrizDenavitHartenberg(q2, 0, l2, 0);
A23 = matrizDenavitHartenberg(q3, 0, l3, 0);
```

Con las coordenadas obtenidas, el script forma:

```
Pd = [xd;
      yd;
      zd];
```

Q conocido → cinematica_directa_3R_DH.m → Pd

El Pd : Punto cartesiano deseado calculado es

X= 0.3116

Y= 0.1799

Z= 0.8232

5. Segunda parte: la semilla q0

Ahora se elige una postura inicial diferente de la postura conocida:

```
q0 = [0;
      10;
      10];
```

q0 es la semilla desde la cual comienza la búsqueda numérica. Es importante distinguirla de q1_real, q2_real y q3_real: estos últimos sirvieron para generar Pd; q0 es solamente el punto de partida del algoritmo iterativo.

6. Llamada a **cinematica_inversa_3R_numerica.m**

El programa de prueba llama a la cinemática inversa mediante:

```
[q,success,iter] = cinematica_inversa_3R_numerica(Pd,q0);
```

Entradas reales de esta función: Pd, el punto deseado [xd; yd; zd] en metros, y q0, la postura inicial [Q1; Q2; Q3] en grados:

```
q0 = [0;
      10;
      10];
```

Salidas reales: q, la solución articular encontrada; success, que vale 1 si converge y 0 si no converge; e iter, que informa el número de iteraciones realizadas.

7. ¿Qué hace internamente cinematica_inversa_3R_numerica.m?

La función utiliza los parámetros lambda=0.01, tol_pos=1e-5, max_iter=200 y alpha=1.0. Inicializa q=q0 y repite el siguiente ciclo:

Paso	Operación real del programa
1. Cinemática directa	Llama a cinematica_directa_3R_DH(q(1),q(2),q(3)) y forma Pactual=[x;y;z].
2. Error cartesiano	Calcula e = Pd - Pactual.
3. Convergencia	Si norm(e) < tol_pos, asigna success=1 y termina.
4. Jacobiano	Llama a jacobiano_3R(q(1),q(2),q(3)).
5. Corrección DLS	Llama a correccion_DLS_3R(J,e,lambda).
6. Nueva postura	Actualiza q = q + alpha*delta_q y vuelve a comenzar.

Pd → CD → Pactual → e → J → DLS → ΔQ → Q nueva → repetir

8. Función jacobiano_3R.m

Esta función recibe q1, q2 y q3 en grados y devuelve J, el Jacobiano de posición 3×3. En el archivo se calculan primero las expresiones auxiliares A y B, luego Jrad, y finalmente se convierte el Jacobiano para trabajar con incrementos articulares expresados en grados:

```
J = (pi/180)*Jrad;
```

Por lo tanto, su papel dentro de la inversa es describir localmente cómo pequeños cambios de Q1, Q2 y Q3 producen cambios en x, y, z.

9. Función correccion_DLS_3R.m

Recibe J, el error cartesiano e y el factor lambda. Devuelve delta_q=[dQ1;dQ2;dQ3] en grados. La operación implementada es:

```
delta_q = J' * ((J*J' + lambda^2*I) \ e);
```

Su misión es convertir el error cartesiano en una corrección articular. Esa corrección se suma a la postura actual para generar la siguiente aproximación.

10. Árbol real de llamadas

```
prueba_cinematica_inversa_3R.m
│
├── cinematica_directa_3R_DH.m       % genera inicialmente Pd
│   └── matrizDenavitHartenberg.m
│
└── cinematica_inversa_3R_numerica.m
    │
    ├── cinematica_directa_3R_DH.m
    │   └── matrizDenavitHartenberg.m
    │
    ├── jacobiano_3R.m
    │
    └── correccion_DLS_3R.m

Después de converger:
prueba_cinematica_inversa_3R.m
└── cinematica_directa_3R_DH.m       % verificación final
```

11. Verificación final realizada por el programa de prueba

Una vez obtenida q, el script no se limita a aceptar la respuesta. Vuelve a introducir la solución en la cinemática directa:

```
[xv,yv,zv] = cinematica_directa_3R_DH(q(1),q(2),q(3));

Pverificado = [xv;
               yv;
               zv];
```

Finalmente muestra:

```
disp('Error final:')
disp(Pd-Pverificado)
```

Si success vale 1 y Pd-Pverificado es prácticamente cero, el conjunto ha superado la prueba: la solución articular encontrada por la cinemática inversa lleva nuevamente al punto cartesiano deseado.

12. Idea clave de la prueba

La postura conocida y la semilla cumplen funciones diferentes. La postura conocida q1_real, q2_real, q3_real sirve para crear un Pd alcanzable. La semilla q0 indica desde dónde comienza a buscar la cinemática inversa. La solución q encontrada puede coincidir con la postura original o puede ser otra configuración articular que alcance el mismo Pd.

No estamos comprobando que Q final sea obligatoriamente igual a Q original.

Estamos comprobando que la Q encontrada produzca el mismo Pd.

13. Programa de prueba utilizado

```
clc
clear

q1_real = 30;
q2_real = 30;
q3_real = 30;

[xd,yd,zd] = cinematica_directa_3R_DH(q1_real,q2_real,q3_real);

Pd = [xd;
      yd;
      zd];

q0 = [0;
      10;
      10];

[q,success,iter] = cinematica_inversa_3R_numerica(Pd,q0);

[xv,yv,zv] = cinematica_directa_3R_DH(q(1),q(2),q(3));

Pverificado = [xv;
               yv;
               zv];

error_final = Pd-Pverificado;
```

14. Resumen final

Nombre real	Responsabilidad
prueba_cinematica_inversa_3R.m	Ejecuta y verifica el ensayo completo.
cinematica_directa_3R_DH.m	Q1,Q2,Q3 → x,y,z.
matrizDenavitHartenberg.m	theta,d,a,alpha → matriz A 4×4.
cinematica_inversa_3R_numerica.m	Pd,q0 → q,success,iter mediante iteraciones.
jacobiano_3R.m	Q1,Q2,Q3 → J 3×3.

correccion_DLS_3R.m	J,e,lambda → delta_q.

Relación entre Cinemática inversa

Entre la Cinemática Inversa y Damped Least Squares, DLS

(Damped Least Squares, DLS) **correccion_DLS_3R.m**

$$\Delta Q = J^T \cdot (J \cdot J^T + \lambda^2 \cdot I)^{-1} \cdot e$$

¿Cómo funciona nuestra cinemática inversa numérica 3R?

Del programa de prueba hasta la convergencia: Jacobiano, ΔQ, DLS, λ, α y tolerancia

Esta guía sigue exactamente la lógica de los programas que estamos utilizando con nuestro robot 3R (l1 = 0,50 m, l2 = 0,30 m, l3 = 0,20 m). Los ángulos Q1, Q2 y Q3 se manejan en grados.

1. Empezamos por el programa de prueba

El programa prueba_cinematica_inversa_3R.m parte de unos ángulos conocidos Q1, Q2 y Q3. Esos valores se usan solamente para fabricar un punto cartesiano que sabemos que está dentro del alcance del robot.

```
q1_real = 30;
q2_real = 30;
q3_real = 30;

[xd,yd,zd] = cinematica_directa_3R_DH(q1_real,q2_real,q3_real);

Pd = [xd; yd; zd];
```

Así obtenemos el punto deseado Pd = [xd; yd; zd]. Luego elegimos una semilla distinta q0. La cinemática inversa no recibe los ángulos reales anteriores: recibe Pd y una postura inicial desde la cual comenzar a buscar.

```
q0 = [0; 10; 10];

[q,success,iter] = cinematica_inversa_3R_numerica(Pd,q0);
```

2. ¿Qué debe encontrar la cinemática inversa?

Debe encontrar valores de Q1, Q2 y Q3 que coloquen el extremo del robot en Pd. Como el método es iterativo, empieza con q0 y va corrigiendo los ángulos poco a poco.

$$Q = [Q1; Q2; Q3]$$

3. Primera operación: cinemática directa

Con los valores actuales de Q1, Q2 y Q3, la rutina cinematica_directa_3R_DH.m calcula dónde está realmente la punta.

$$Q \rightarrow \text{Cinemática directa} \rightarrow Pactual = [x; y; z]$$

La cinemática directa utiliza matrizDenavitHartenberg.m para construir las matrices homogéneas del robot.

4. Calculamos el error cartesiano

$$e = Pd - Pactual$$

El vector e tiene tres componentes: error en X, error en Y y error en Z. Si e todavía es grande, debemos modificar Q1, Q2 y Q3.

$$e = [ex; ey; ez]$$

5. ¿Qué aporta el Jacobiano?

La rutina jacobiano_3R.m calcula el Jacobiano en la postura actual. El Jacobiano relaciona pequeñas variaciones articulares con pequeñas variaciones cartesianas.

$$\Delta P = J \cdot \Delta Q$$

Aquí ΔQ = [ΔQ1; ΔQ2; ΔQ3]. Por lo tanto, J nos dice cómo se moverán X, Y y Z si hacemos pequeños cambios en Q1, Q2 y Q3.

6. Si pudiéramos invertir J perfectamente

Como en la cinemática inversa conocemos el error cartesiano y queremos obtener la corrección articular, idealmente podríamos escribir:

$$e \approx J \cdot \Delta Q$$

$$\Delta Q = J^{-1} \cdot e$$

Esta sería la forma directa: tomar el error cartesiano e y transformarlo en una corrección de Q1, Q2 y Q3.

7. ¿Por qué no usamos simplemente J^{-1}?

Porque el Jacobiano puede llegar a una configuración singular o estar cerca de ella. También puede estar mal condicionado: una variación cartesiana muy pequeña puede producir una corrección articular muy grande. En esas situaciones, una inversión directa puede ser numéricamente inestable.

$$\text{error cartesiano pequeño} \rightarrow \Delta Q \text{ excesivamente grande}$$

Por eso usamos una solución amortiguada, en lugar de depender de la inversa directa del Jacobiano.

8. La corrección DLS y el cálculo de ΔQ

La cinemática inversa llama a **correccion_DLS_3R.m**. Esta función recibe J, e y λ, y devuelve ΔQ.

$$\Delta Q = J^T \cdot (J \cdot J^T + \lambda^2 \cdot I)^{-1} \cdot e$$

```
delta_q = J' * ((J*J' + lambda^2*I) \ e);
```

Esta expresión es la solución de mínimos cuadrados amortiguados (Damped Least Squares, DLS). El criterio de mínimos cuadrados está incorporado en esta fórmula: no aparece como un paso separado del programa.

9. ¿Dónde aparecen los mínimos cuadrados?

La idea matemática es buscar una corrección ΔQ que reduzca el error cartesiano, evitando al mismo tiempo correcciones articulares exageradas. Una forma de expresar ese compromiso es:

$$\text{minimizar: } \|J \cdot \Delta Q - e\|^2 + \lambda^2 \|\Delta Q\|^2$$

El primer término intenta que J·ΔQ se parezca al error e que queremos corregir. El segundo término penaliza una corrección ΔQ demasiado grande. De este criterio se obtiene la expresión DLS que usamos en correccion_DLS_3R.m.

10. ¿Qué hace λ?

```
lambda = 0.01;
```

λ es el factor de amortiguamiento. En nuestro programa vale 0,01. Su presencia en λ²·I regulariza el cálculo y ayuda a evitar que ΔQ se dispare cuando el Jacobiano está cerca de una singularidad o mal condicionado.

$$J \cdot J^T \rightarrow J \cdot J^T + \lambda^2 \cdot I$$

DLS no elimina físicamente la singularidad del robot; hace más estable el cálculo numérico alrededor de esas situaciones.

11. Ya tenemos ΔQ: actualizamos los ángulos

Una vez calculado ΔQ, la cinemática inversa modifica los valores actuales de Q1, Q2 y Q3.

$$Q_{nuevo} = Q_{actual} + \alpha \cdot \Delta Q$$

```
q = q + alpha*delta_q;
```

En nuestro programa α = 1,0. Por lo tanto usamos toda la corrección calculada. Si α fuese menor que 1, aplicaríamos solamente una fracción de ΔQ.

$$Q1_nuevo = Q1 + \alpha \cdot \Delta Q1$$
$$Q2_nuevo = Q2 + \alpha \cdot \Delta Q2$$
$$Q3_nuevo = Q3 + \alpha \cdot \Delta Q3$$

12. Con los nuevos Q volvemos a calcular X, Y, Z

Los nuevos Q1, Q2 y Q3 vuelven a entrar en la cinemática directa. Así obtenemos una nueva posición Pactual.

$$Qnuevo \rightarrow \text{Cinemática directa} \rightarrow Pactual_nuevo$$

Entonces volvemos a calcular:

$$e_nuevo = Pd - Pactual_nuevo$$

Si todavía existe un error apreciable, calculamos nuevamente el Jacobiano, volvemos a obtener ΔQ mediante DLS, actualizamos Q y repetimos el ciclo.

13. El IF que decide cuándo terminamos

En cinematica_inversa_3R_numerica.m la tolerancia de posición está definida como:

```
tol_pos = 1e-5;
```

y se verifica:

```
if norm(e) < tol_pos
    success = 1;
    return
end
```

Cuando la norma del error es menor que 10^{-5} m, consideramos que el punto fue alcanzado con suficiente precisión y la función termina con success = 1.

Si no alcanza esa tolerancia, continúa iterando. Para evitar un ciclo indefinido existe además:

```
max_iter = 200;
```

Si se completan 200 iteraciones sin satisfacer la tolerancia, la función termina sin declarar convergencia.

14. El ciclo completo

$$Q \rightarrow CD \rightarrow Pactual \rightarrow e \rightarrow J \rightarrow DLS(\lambda) \rightarrow \Delta Q \rightarrow Qnuevo \rightarrow CD \rightarrow \ldots$$

Este es el corazón de nuestra cinemática inversa numérica. Cada vuelta intenta reducir el error y acercar la punta del robot al punto Pd.

15. ¿Qué función controla todo?

La función principal que controla este proceso es cinematica_inversa_3R_numerica.m. Ella llama a las demás rutinas en el orden necesario.

```
cinematica_inversa_3R_numerica.m
    |
    +-- cinematica_directa_3R_DH.m
    |       |
    |       +-- matrizDenavitHartenberg.m
    |
    +-- jacobiano_3R.m
    |
    +-- correccion_DLS_3R.m
            |
            +-- calcula delta_q mediante DLS
```

16. Qué nos muestra el programa de prueba

prueba_cinematica_inversa_3R.m muestra el punto cartesiano deseado, los ángulos encontrados, si hubo convergencia (success), cuántas iteraciones fueron necesarias (iter), y finalmente verifica la solución haciendo otra cinemática directa.

$$\text{Q encontrado} \rightarrow \text{Cinemática directa} \rightarrow \text{Pverificado}$$

$$\text{Error final} = \text{Pd} - \text{Pverificado}$$

Si success = 1 y el error final es prácticamente cero, la cadena completa está funcionando.

17. Resumen para recordar

1) Tengo un Pd deseado.
2) Parto de una semilla Q.
3) Calculo Pactual.
4) Calculo e.
5) Calculo J.
6) DLS usa J, e y λ para obtener ΔQ.
7) Actualizo Q = Q + αΔQ.
8) Vuelvo a calcular Pactual.
9) Repito hasta que ||e|| < tol_pos.

La razón fundamental de utilizar DLS es que queremos transformar el error cartesiano en una corrección articular de manera estable. Los mínimos cuadrados buscan una buena corrección del error; el término con λ evita que esa corrección articular se vuelva excesiva cerca de singularidades.